

Fakulta informatiky a informačných technológií STU

Ilkovičova 2, 842 16 Karlova Ves, Bratislava

Princípy počítačovej grafiky a spracovania obrazu

Semestrálny projekt

Jakub Jediný, 120948

Andrej Krivošík, 120955

Ut. 18:00, Ing. Peter Kapec, PhD.

2025/2026

Obsah

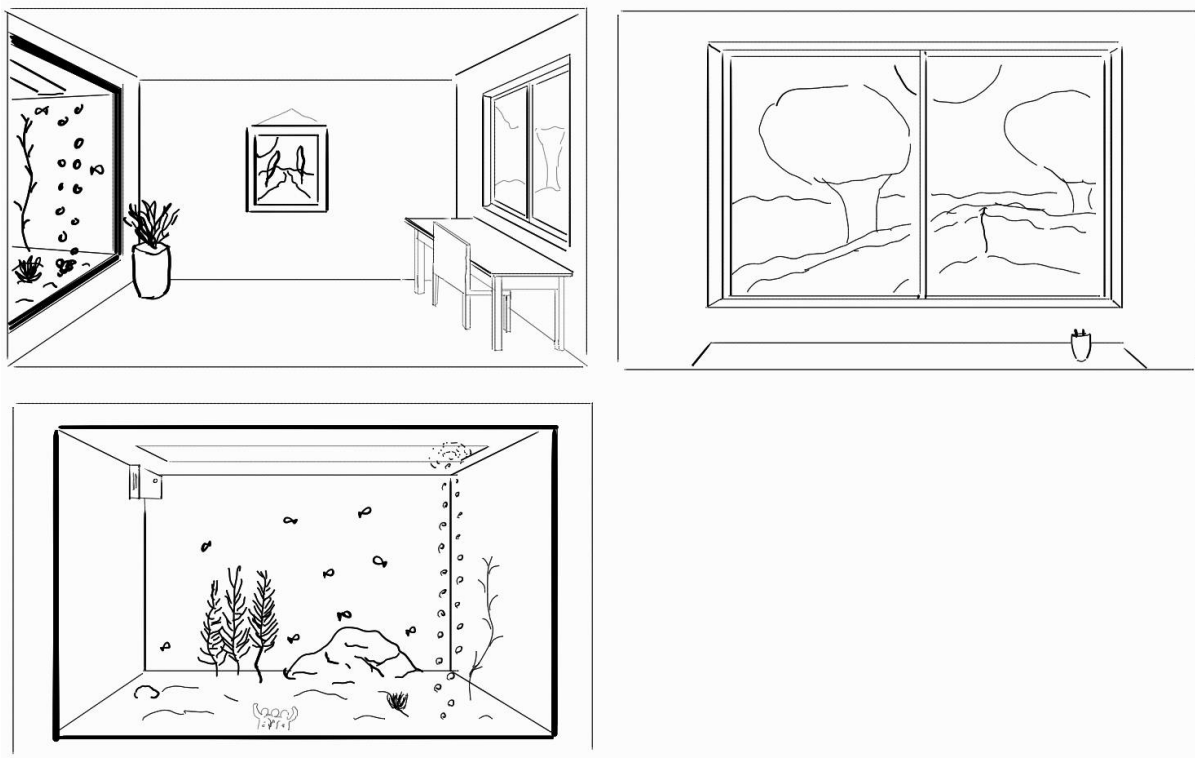
Návrh projektu.....	3
Dátové štruktúry.....	4
Algoritmy	5
Scény.....	9
Objekty	10
Obrázky z projektu.....	11

Návrh projektu

V prvej scéne vchádza kamera do miestnosti, na ľavej strane sa nachádza v stene akvárium. Na pravej strane sa nachádza stolička a písací stôl, na ktorom sa nachádzajú knihy, papiere, perá a lampa. Za stolíkom sa nachádza okno, na ktoré sa kamera zameria a budeme môcť vidieť zmenu dňa na noc (slnko->zdroj svetla sa bude hýbať prejde z východu slnka, až po západ a vyjde mesiac). Pohyb slnka spôsobí zmenu tieňov na objektoch vonku (strom, plot, rastlinky/kríky).

V druhej scéne kamera preletí cez celé akvárium. Vstúpi v ľavej časti, potom preletí akváriom pozdĺž, a vystúpi v pravej časti. Kamera sa celý čas bude pomaly točiť a bude zaznamenávať rôzne objekty v akváriu (plávanie rýb, pohyb rastlín, bublinky vychádzajúce s filtra, presklené farebné sošky), v akváriu je zem z piesku a na nej sa nachádzajú kamienky a mušle. Jeden z objektov môže byť aj ryba, ktorá sa bude snažiť zakopať sa do piesku a bude tak vytvárať častice piesku. Scéna bude obsahovať aj ako dve ryby narazia do seba počas plávania (collisions). Akvárium bude celý čas mierne osvetlené LED-kovým svetlom. Scéna začína a končí vonku z akvária, teda kamera bude prechádzať cez sklo.

V tretej scéne sa kamera opäť zameria na okno, kde je možné vidieť z noci na deň. Následne sa kamera otáča späť na akvárium, a je možné pozorovať naturálny pohyb objektov v akváriu. Následne sa kamera otáča doľava a vychádza z miestnosti s postupným otočením o 180 stupňov, kde znovu zachytí celú miestnosť a končí rovnako ako prvá scéna začína.



Obrázok 1: Návrhy scén

Štruktúra projektu

Projekt je implementovaný ako jedna OpenGL aplikácia, kde je hlavná riadiaca logika sústredená v súbore main.cpp. Ostatné časti projektu sú rozdelené do samostatných častí podľa funkčnosti.

main.cpp

Súbor main.cpp predstavuje vstupný bod celej aplikácie a riadi jej životný cyklus.

Jeho hlavné úlohy sú:

- inicializácia knižníc GLFW a GLEW
- vytvorenie okna a OpenGL kontextu
- registrácia vstupných callbackov (myš, klávesnica, resize okna)
- vytvorenie a inicializácia všetkých objektov a systémov
- riadenie hlavnej slučky aplikácie (update + render)

V hlavnej slučke sa postupne vykonáva aktualizácia času, pohybu kamery, fyzikálnych simulácií, časticového systému a samotné renderovanie scény.

Kamera

Kamera je riešená ako samostatný modul, ktorý uchováva:

- pozíciu kamery v priestore
- smer pohľadu
- orientáciu pomocou uhlov yaw a pitch

Kamera poskytuje view maticu, ktorá sa používa pri renderovaní všetkých objektov v scéne.

Objekty scény

Všetky vizuálne objekty v scéne sú reprezentované triedou T_object.

Každý objekt obsahuje:

- geometrický model načítaný zo súboru .obj
- textúru alebo sadu textúr
- materiálové vlastnosti
- transformačné informácie

T_objekt slúži ako jednotné rozhranie na vykresľovanie objektov a je používaný počas bežného render passu aj počas depth passu pri výpočte tieňov.

Graf scény a hierarchia objektov

Priestorové usporiadanie objektov v scéne je riešené pomocou jednoduchéj hierarchickej štruktúry (grafu scény). Každý uzol grafu obsahuje:

- lokálnu transformačnú maticu
- referenciu na objekt
- zoznam potomkov

Takýto prístup umožňuje:

- hierarchické transformácie objektov
- jednoduché zoskupovanie objektov
- rekurzívne renderovanie celej scény

Dátové štruktúry a Algoritmy

Časticový systém

Použitý časticový systém je klasický CPU-driven particle system s GPU renderingom. Je navrhnutý tak aby simuloval zrnká piesku vyhadzované smerom hore a padajúce v smere prúdenia vody.

1. Základná štruktúra častice:

```
struct Particle {  
    glm::vec3 pos;  
    glm::vec3 vel;  
    glm::vec3 color;  
    float age = 0.0f;  
    float lifetime = 0.05f;  
    float mass = 1.0f;  
};
```

Každá častica má pozíciu, rýchlosť, farbu, vek a životnosť.

Hmotnosť sa používa na ovplyvnenie reakcie na gravitáciu a odpor.

2. Emisia častíc:

```
float spawnInterval = 1.1f;  
int spawnPerBurst = 300;
```

Každých 1.1 sekundy sa zjaví 300 častíc naraz, toto tvorí taký menší „výbuch“ alebo „puff“ efekt.

V `update()` sa počíta `spawnTimer += dt`, a keď presiahne `spawnInterval`, zavolá sa `emitBurst()`

V `emitBurst()` sa 300-krát vytvorí nová častica, ktorej rýchlosť je náhodná smerom hore a mierne do strán. Takisto každá častica má farbu od čiernej po svetlo šedú, ale je väčšia pravdepodobnosť, že bude farba svetlejšia aby sa podobala na farbu piesku na dne.

3. Fyzikálna aktualizácia:

```
glm::vec3 gravity(0.0f, -0.5f, 0.0f);    -> vplyv gravitácie  
glm::vec3 water(0.0f, 0.0f, -2.0f);     -> vplyv prúdenia vody
```

Pre každú časticu sa aplikuje:

$p.vel += (gravity + water) / p.mass * dt$; $\rightarrow F = ma \Rightarrow a = F/m$

```
p.vel *= 0.95f;          -> odpor vody (drag)
p.pos += p.vel * dt;
p.age += dt;             -> Keď p.age >= p.lifetime -> častica sa vymaže
```

4. Rendering:

Používame GL_POINTS, takže každá častica vyzerá ako malá guľôčka.

Vertex shader:

```
gl_PointSize = 6.0 * (1.0 / clipPos.w);
```

-> Čím je častica ďalej od kamery, tým menšia je na obrazovke.

Post-Processing Blur Filter

Je to jednoprechodový 3x3 box blur aplikovaný na celú obrazovku. Je to „Gaussian-like“ blur, ktorý sa dá urobiť v reálnom čase.

1. Vyrendrovanie do FBO:

```
PostProcess postProcess(SCR_WIDTH, SCR_HEIGHT, "blur_shader.vs", "blur_shader.fs");
```

Vytvorí sa Framebuffer Object s jednou farebnou textúrou (texColorBuffer) o veľkosti obrazovky.

Všetky objekty, častice, tieň atď. sa renderujú do tohto FBO namiesto priamo na obrazovku.

2. Post-processing pass:

```
postProcess.UnbindFramebuffer();    -> vrátenie sa na default framebuffer (obrazovka)
```

```
if (postProcessEnabled) {
```

```
    glClear(GL_COLOR_BUFFER_BIT);
```

```
    postProcess.Render();            -> spustenie blur filtru
```

```
}
```

3. Shader(blur_shader.fs):

```
#version 330 core
```

```
out vec4 FragColor;
```

```
in vec2 TexCoords;
```

```
uniform sampler2D screenTexture;
```

```
uniform float offset = 1.0 / 500.0;
```

```
void main() {
```

```
    vec3 result = vec3(0.0);
```

```
    for(int x = -1; x <= 1; ++x)
```

```
        for(int y = -1; y <= 1; ++y)
```

```
            result += texture(screenTexture, TexCoords + vec2(x, y) * offset).rgb / 9.0;
```

```
    FragColor = vec4(result, 1.0);
```

```
}
```

offset = 1.0 / 500.0 -> Definuje výraznosť blur efektu
Dva vnorené cykly -> Pre aktuálny pixel sa nasampluje 9 susedných pixlov (3×3 okolie) vrátane seba samého.
texture() -> Posunie UV súradnice o ±offset vľavo/vpravo/hore/dole.
/9.0 -> Priemer z 9 vzoriek(box filter)

Výsledok sa zapíše ako finálna farba pixelu, takže každý pixel na obrazovke sa nahradí priemernou farbou svojho 3×3 okolia.

4. Výsledný vizuálny efekt:

Blur efekt simuluje rozmazanie ako pod vodou.

Ostré hrany (napr. tiene, obrysy objektov) sa zmäknú.

Svetlé miesta sa rozmažú, takže vzniká jemný glow efekt.

Rigid Body Dynamics

Je to fyzikálny simulátor tuhého telesa s rotáciou, krútiacim momentom, odporom vody a kolíziami.

1. Štruktúra Rigidbody:

```
struct Rigidbody {  
    glm::vec3 position;  
    glm::vec3 velocity = glm::vec3(0);  
    glm::vec3 angularVelocity = glm::vec3(0);  
    glm::vec3 rotation = glm::vec3(-90,0,-90);  
  
    glm::vec3 force = glm::vec3(0);  
    glm::vec3 torque = glm::vec3(0);  
  
    float mass = 1.0f;  
    float restitution = 0.6f;  
    float drag = 0.98f;  
};
```

Táto štruktúra predstavuje potápača ako jedno fyzikálne teleso v scéne.

Najdôležitejšie parametre sú: aktuálna poloha potápača, rýchlosť, a uhlová rýchlosť, ktorá určuje, ako rýchlo a okolo ktorej osi sa točí.

Následne štruktúra používa dočasné premenné pre sily a krútiaci moment(force, torque). Tieto hodnoty sa každý frame setnú a na konci frame sa vymažú.

Nakoniec tri dôležité vlastnosti: hmotnosť (mass), koeficient odrazu (restitution) – ako veľmi sa odrazí od dna a odpor vody (drag).

2. Update:

```
void update(float dt) {  
    //1. Lineárny pohyb  
    glm::vec3 accel = force / mass;  
    velocity += accel * dt;  
    velocity *= drag;      -> odpor vody  
    position += velocity * dt;
```

```
// 2. Uhlový pohyb
glm::vec3 angularAccel = torque;
angularVelocity += angularAccel * dt;
angularVelocity *= 0.99f;
rotation += glm::degrees(angularVelocity) * dt;

// 3. Reset síl
force = glm::vec3(0);
torque = glm::vec3(0);
}
```

1. Lineárny pohyb (posun v priestore):

Vypočíta aktuálne zrýchlenie ako celkovú pôsobiacu silu delenú hmotnosťou a pridá toto zrýchlenie k aktuálnej rýchlosti.

Aplikuje odpor vody – rýchlosť sa každý tick mierne zníži násobením koeficientom drag, čo vytvára efekt tlmenia vo vode.

Na základe novej rýchlosti posunie objekt v priestore.

2. Uhlový pohyb (otáčanie)

Vypočíta uhlové zrýchlenie priamo z aktuálneho krútiaceho momentu.

Pridá uhlové zrýchlenie k uhlovej rýchlosti.

Simuluje odpor prostredia na otáčanie – uhlová rýchlosť sa každý tick trochu spomalí.

Aktualizuje samotnú rotáciu objektu – prevedie uhlovú rýchlosť na stupne a pridá ju k aktuálnej orientácii.

3. Vyčistenie síl

Po spracovaní vynuluje všetky nahromadené sily a krútiace momenty.

3. Aplikovanie síl:

```
void applyForce(const glm::vec3& f, const glm::vec3& atPoint = glm::vec3(0)) {
    force += f;
    if (atPoint != glm::vec3(0)) {
        torque += glm::cross(atPoint - position, f);
    }
}
```

Ak sa sila aplikuje priamo do ťažiska (atPoint = 0), objekt sa len posunie, ale ak sa sila aplikuje mimo ťažiska, vznikne krútiaci moment -> potápač sa začne otáčať.

4. Kolízia s podlahou:

```
void checkFloorCollision() {
    if (position.y < 0.1f) {
        position.y = 0.1f;
        velocity.y = -velocity.y * restitution;           -> tlmený odraz
        glm::vec3 impactOffset = glm::vec3(0.5f, 0, 0);
        applyForce(glm::vec3(0, 60.0f, 0), position + impactOffset);
    }
}
```

```

    }
}

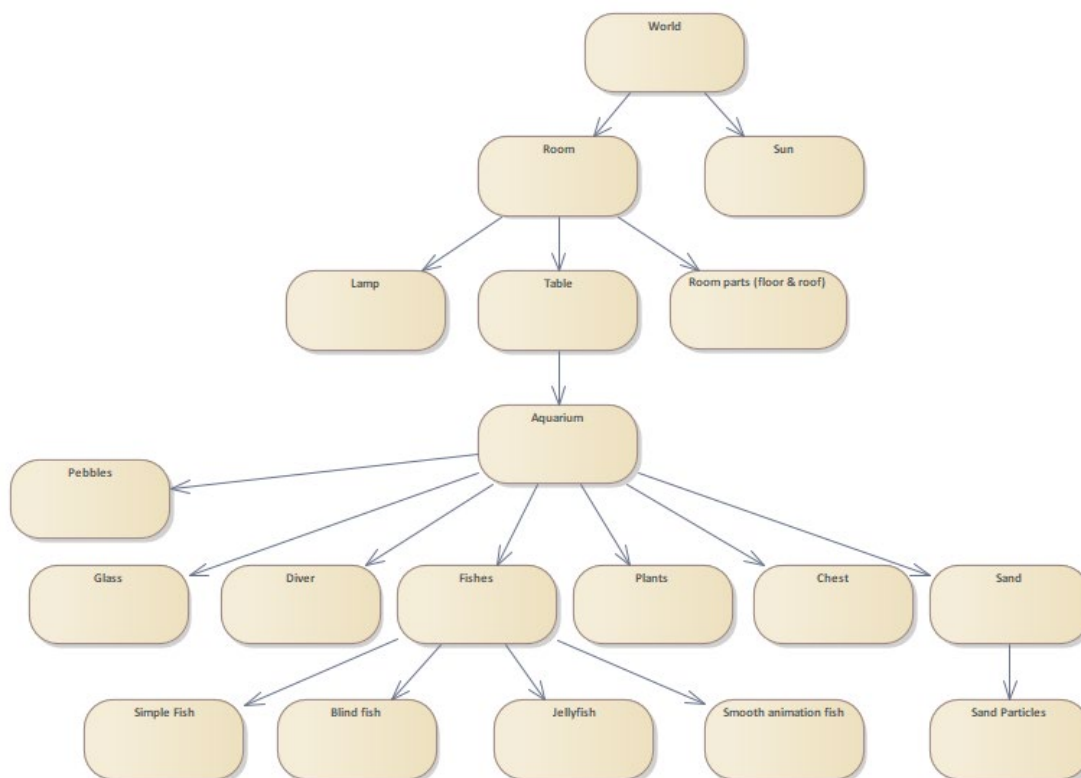
```

5. Vizualizácia:

V scéne potápač začne padať a keď narazí na zem, tak sa odrazí a točí sa okolo vlastnej osi a pri tom ako znovu padá jeho rýchlosť sa spomalí kvôli odporu vody.

Scény

Scéna



Obrázok 2: Graf scény

3D svet scény sa skladá zo slnka, ktoré vrhá svetlo cez okno miestnosti.

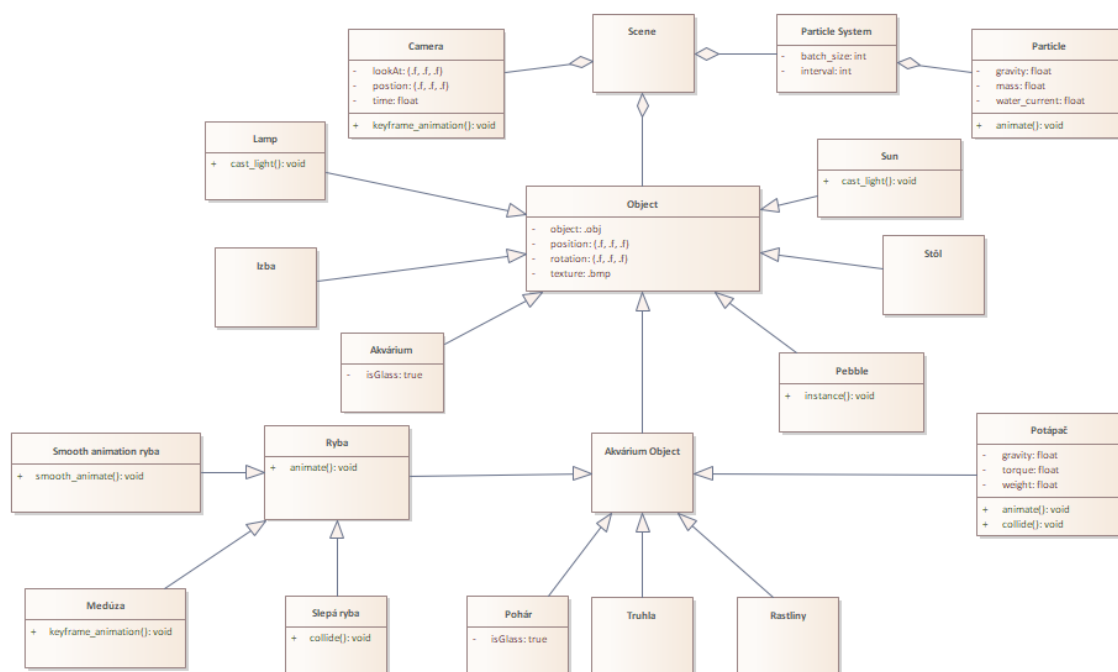
V miestnosti sa nachádza lampa, vrhajúca svetlo do každého rohu miestnosti, stôl na ktorom sa nachádza akvárium a stolová lampa, ktorá reprezentuje spotlight typ svetla.

V akváriu vieme nájsť pohár, ktorý slúži na ukázanie správneho vykresľovania viacerých priesvitných objektov, potápača, ktorý začína padať na spodok akvária, od ktorého sa následne odráža a skáče smerom hore, pričom sa pomaly otáča okolo svojej osi. Ďalšími objektami sú rôzne rastlinky a truhla, tieto objekty fungujú iba ako dekorácie. Ďalej máme v akváriu piesok a jeho častice vytvárané

časticovým systémom. Nakoniec máme v akváriu rôzne ryby. Dve ryby sú označené ako simple fish, tieto ryby vykonávajú iba jednoduchú animáciu, pričom pohyb jednej z nich simuluje rybu, ktorá sa chce zahrabať do piesku a tým vyhadzuje častice piesku, ktoré potom padajú na dno pod vplyvom gravitácie a prúdenia vody. Následne vieme v akváriu nájsť „slepú rybu“, ktorá pláva náhodným smerom a odráža sa od stien akvária. Ďalej sa v akváriu nachádza medúza, ktorá sa hýbe v kruhu na základne kľúčových snímok. Nakoniec sa v akváriu nachádza ryba, ktorá vykonáva hladkú animáciu, plávajúca v kruhu so zmenou smeru hore a dole.

V izbe sa nachádza ešte jedno veľké akvárium oproti oknu, ktoré sa procedurálne napĺňa rastlinami, rybami a kameňkami(pebbles).

Objekty



Obrázok 3: Diagram tried objektov

V scéne sa nám nachádza kamera, časticový systém a rôzne objekty.

Kamera vykonáva pohyb pomocou kľúčových snímok, ktoré definujú pozíciu kamery, kam sa má kamera pozeráť a čas za aký sa kamera má do tohto stavu dostať.

Časticový systém definuje pozíciu, rýchlosť, farbu, vek, životnosť a hmotnosť častice. Vyberá pozíciu odkiaľ sa budú častice vytvárať, v akom časovom intervale a koľko častíc sa vytvorí naraz. Každá častica je ovplyvnená gravitáciou a prúdom vody inak, podľa váhy častice.

Každý objekt v scéne je zadefinovaný ako niečo čo sa skladá z .obj a .bmp súborov, v podstate 3D objekt s textúrou. Samozrejme každý objekt má nejakú pozíciu, rotáciu, veľkosť ovplyvnenú škálovaním a nejakú projekciu a view pre kameru.

Objektami v scéne sú slnko a lampa, čo slúžia ako zdroje svetla v našej scéne. Ďalej máme akvárium, ktorého textúra nie je .bmp file, ale má atribút isGlass, čo spôsobí že akvárium je priesvitné a sklo je trochu zafarbené do modra. Takto isto funguje aj objekt poháru, ktorý sa nachádza v akváriu.

Ďalšie objekty v akváriu, sú ryby, každá je animovaná, niektoré ryby používajú jednoduchú animáciu z bodu A do bodu B a naspať s použitím jednoduchých transformácií. Jedna z rýb simuluje „slepú rybu“, v podstate táto ryba má hitbox a pláva náhodne. Pri kolízii so stenou akvária sa vykoná odraz, transformácia rybu a smer akým ryba pláva. Ďalšou rybou je ryba, ktorá využíva catmull-rom splines pre hladkú animáciu. Posledná ryba je medúza, ktorej animácia sa vykonáva na základe kľúčových snímok.

Potápač je objekt určený pre ukázanie rigid body. Initial poloha potápača je na vrchnej časti akvária, odkiaľ padá na dno a pri kolízii sa odráža smerom hore s rotáciou. Pri výpočte polohy potápača sa berie do úvahy gravitácia, odpor vody a krúťivý moment.

Objekt Pebble je v projekte inštancovaný 5000-krát, každý pebble sa vykresluje v súradnicovom ohraničení veľkého akvária s náhodnou rotáciou a scale z definovaných intervalov.

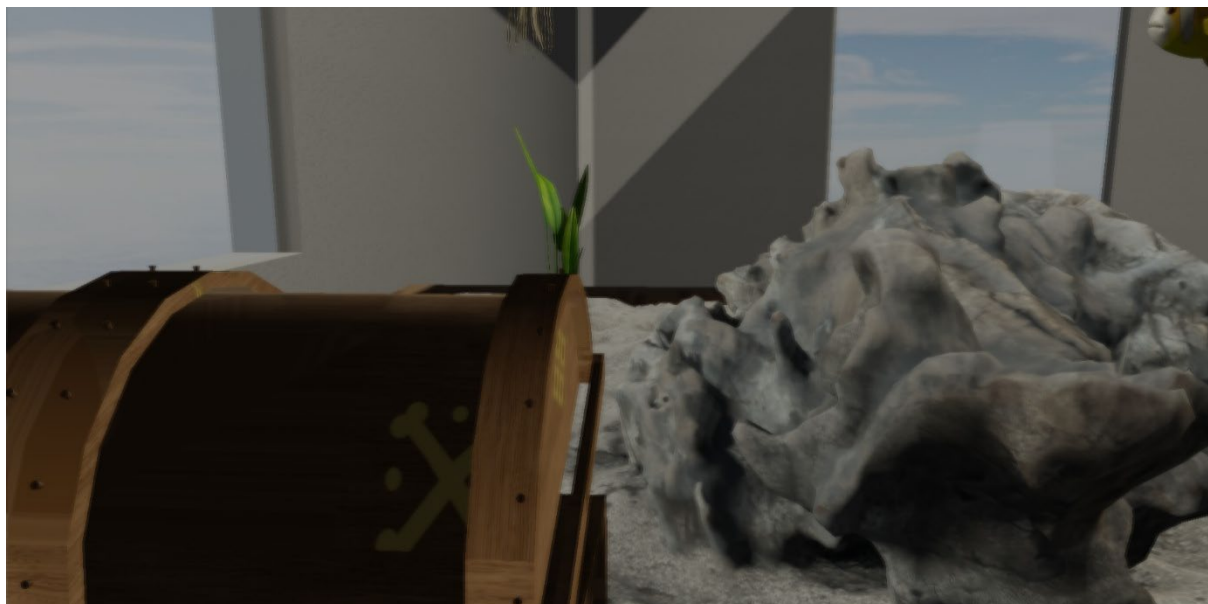
Obrázky z projektu



Obrázok 4: Vstúpenie do miestnosti



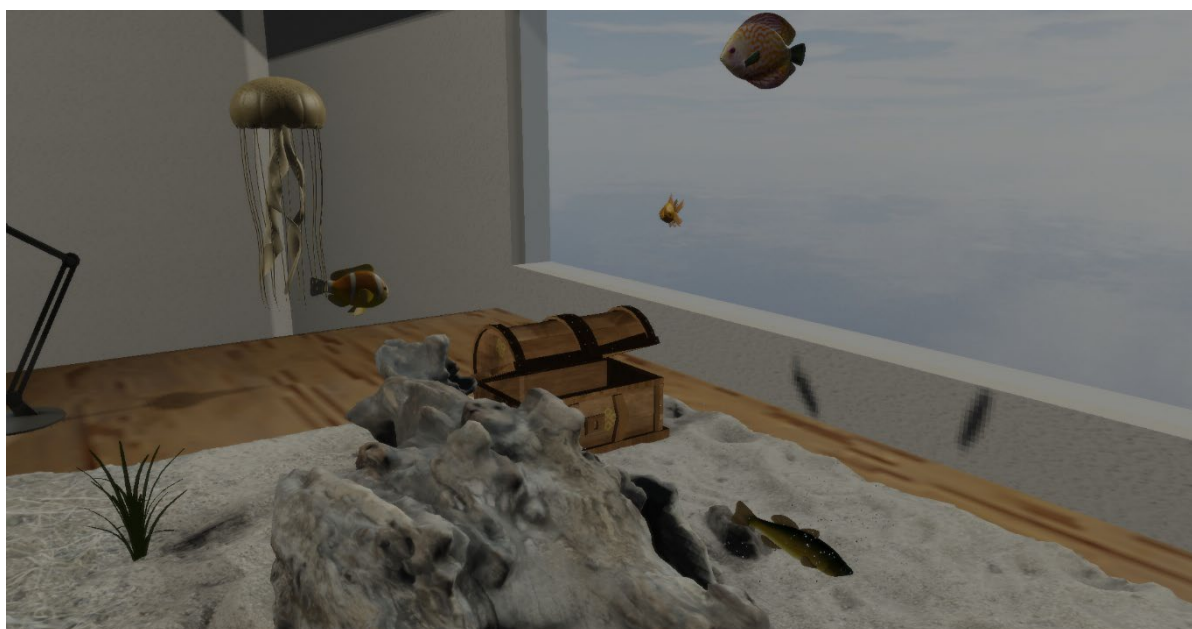
Obrázok 5: Focus na akvárium



Obrázok 6: Vstúpenie do akvária(aplikovanie blur filtru)



Obrázok 7: Focus na rybu zahrabávajúcu sa do piesku



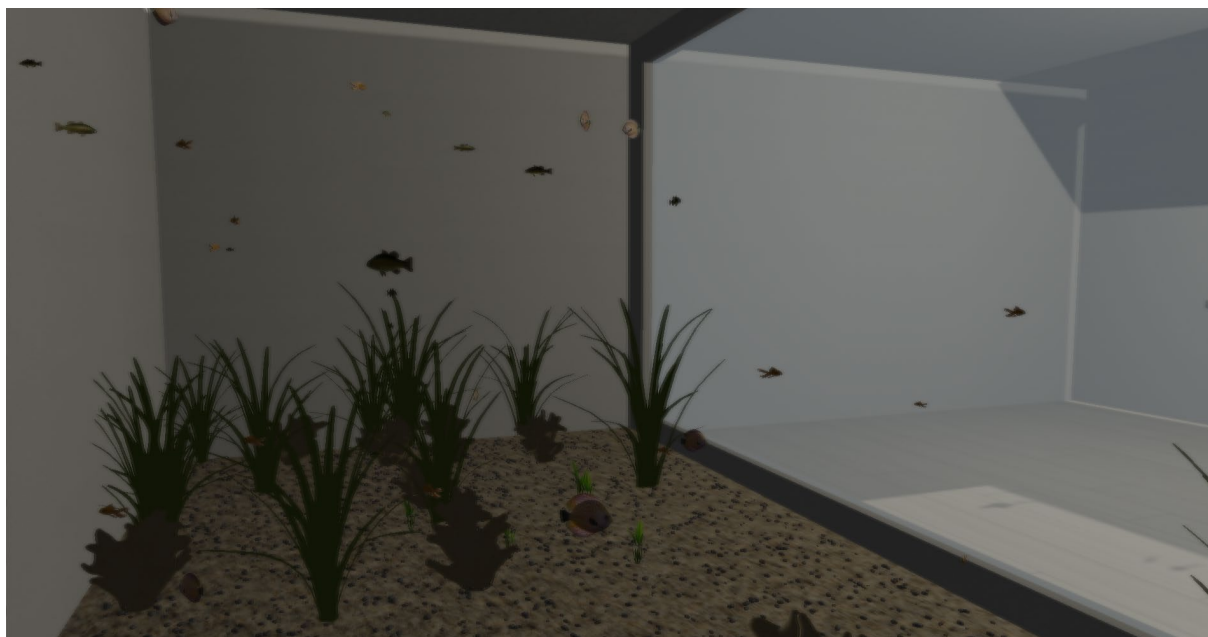
Obrázok 8: Preletenie cez akvárium



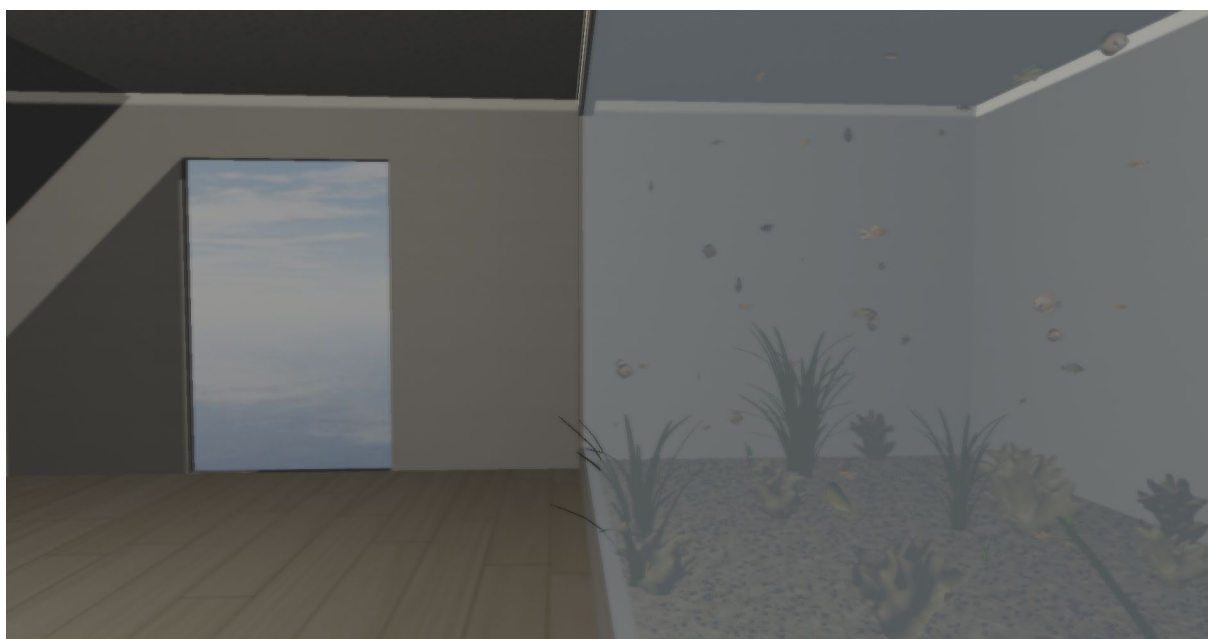
Obrázok 9: Vystúpenie z akvária(deaktivácia blur filtru)



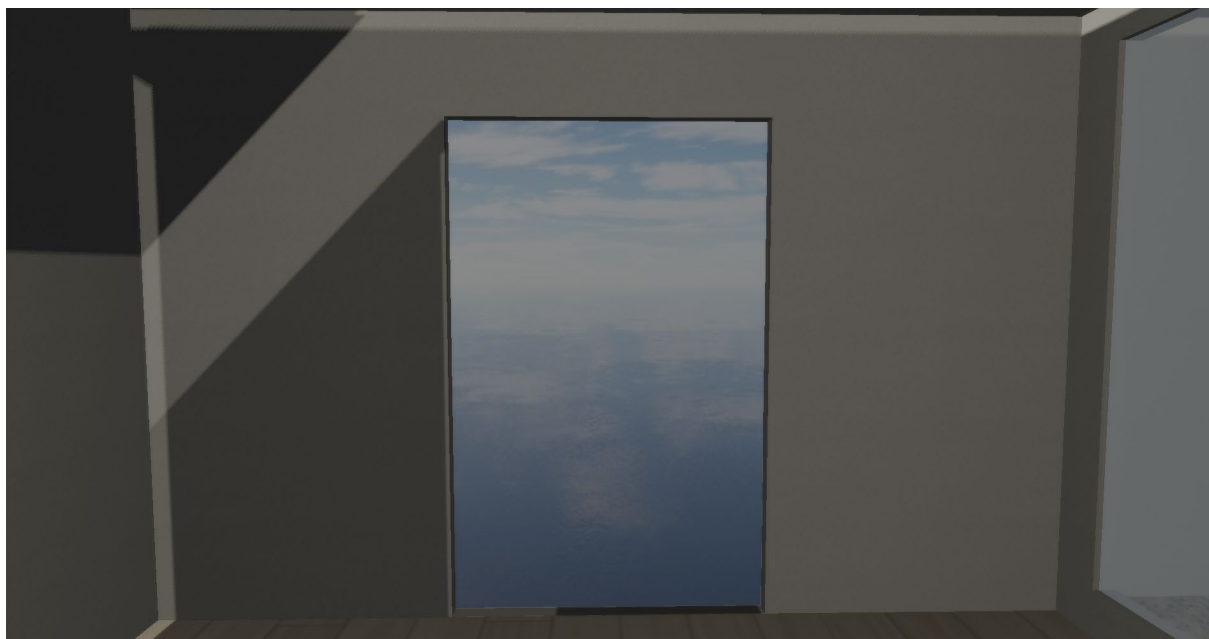
Obrázok 10: Prelet pred akváriom



Obrázok 21: Prelet akváriom



Obrázok 32: Vystúpenie z akvária



Obrázok 43: Odchod z miestnosti